

The use of e-graphs in compilers

Daniel Vella 0147005L

June 26, 2026

1 Introduction

An e-graph is a directed graph designed to represent several equivalent expressions, composed of smaller identical values that are grouped in equivalence classes. In the graph, these are the children of intermediate nodes, known as operators or *e-nodes*, which are applied to the whole class rather than one element at a time to create the next equivalence class.

In [1], Tate et al. instantiated this structure as an E-PEG which is an intermediate representation of a program that is formed by first computing several Program Expression Graph (PEG) for a certain program. A PEG is composed of operator nodes, that are applied to their operand leaves, connected by 'dataflow' edges that illustrate how these operands are used by the operators. After applying some optimisation to the initial program a new PEG is obtained, and these can be merged on the nodes they are equivalent at into an equivalence graph, where each node is an equivalence class of nodes from separate PEG's that have the same function.

This compilation technique is known as *equality saturation*, whereby a source program is converted into an initial e-graph, a set of optimisations are applied, with their results being merged back into the equivalence classes. So the graph gets larger without discarding older versions, as opposed to the traditional destructive optimisation passes where modifications overwrite the source code. The optimisations are then repeated until no further equivalences show up, so the graph would be in a saturated state. Finally a *global profitability heuristic* applies a cost model to the whole graph to extract the most optimal program variation.

This solves an important problem in compiler theory known as the *phase ordering problem*, which occurs when the order of optimisations has an effect on the final version, since some optimisations may prevent the next from operating fully. By evaluating all transformations concurrently, and discarding none of the code this is avoided, making it highly effective for complex algebraic simplifications and instruction selection, among other things.

2 Historical Context

Applying equality saturation to compiler optimisation to solve the *phase ordering problem* was first explored by Tate et al. as described above. However many early frameworks that implemented this had a scalability issue since graph maintenance routines were run after every single rewrite, which led to the number of equivalent structures to grow exponentially. On top of this, the final *global profitability heuristic* required slow, complex mathematical solvers, which needed to be applied to every possible combination derived from the e-graph, making it unrealistic to implement in real-world systems.

This constant need to recalculate profitability was solved by *amortized rebuilding*, a breakthrough published in the `egg` library by Willsey et al. in 2021 [2]. In the same way AVL Trees avoid balancing until the balance factor is larger than 1, avoiding unnecessary computations that will not do much to optimise the tree, *amortized rebuilding* is used to defer graph maintenance routines to scheduled optimization intervals where they are batched together. This seemingly

simple change improved the efficiency of equality saturation enough to transform it from theory into a practical system, in many use cases as discussed in the next section.

3 Real-World Uses

Following the release of this library, previously theoretical implementations of e-graphs were now realistic to build and develop, such as the Herbie framework [3]. This is a framework that avoids precision rounding errors in floating-point math formulas by rewriting them. Until the `egg` library released this was built in a narrow environment that ran inefficiently, but then it was able to be implemented on a larger scale by using this library.

Another form of optimisation that was lifted from theoretical to practical by the `egg` library is that of complex matrix calculations, which are very important for machine learning, due to the calculations run on tensors. In the SPORES system [4], relational equality saturation is used to optimise large scale linear algebra computations. In practice these optimisations allow machine learning algorithms, especially on the larger scales used by large language models, to be computed faster.

4 Current Trends and Future Projections

As time goes on the ease of access of libraries that offer *amortized rebuilding* allows for the integration of e-graphs more and more within modern compiler architecture, for example in Multi-Level Intermediate Representations (MLIR). However, in [5], Merckx et al. discuss building the e-graph natively within the compiler’s IR, rather than operating at a single level of abstraction. This facilitates the maintenance of the e-graph state throughout the compilation flow, allowing for pattern rewriting to be interleaved with other compiler transformations.

Alongside this, industrial adoption has also begun, namely Cranelift’s aegraph [6] aims to implement e-graphs as acyclic graphs to enable some compile-time guarantees ensured by the lack of cycles in the graph. This retains most of the optimisation power, so it is a good trade-off for practical uses outside just theory. However, all of these applications happen on the e-graph, not on the cost analysis of it, which is still an active area. Current cost models are static, so a potential future area of research could be dynamic profiling, potentially even using ML-guided heuristics. Based on all of this it seems that equality saturation may eventually become standard compiler practice, rather than a specialist tool.

5 E-graphs for Task 5

My implementation of task 5 achieves dead-code elimination through live-variable analysis. This type of analysis could be used to remove redundant assignments, which an e-graph of the program could group into equivalence classes to choose one from. However, dataflow analyses like liveness are already solved efficiently by fixpoint iteration like what I implemented.

Having said that it’s interesting to note that an e-graph could be built on the Datalog facts generated by task 5 (the implementation would need to be changed because they are not printed explicitly currently), and rewrite rules could then find simplifications missed by the dataflow passes. Although, for a simple ResIR function, the gain from building and saturating an e-graph would likely be negligible, especially with no complex algebraic expressions present.

References

- [1] R. Tate, M. Stepp, Z. Tatlock, and S. Lerner, “Equality saturation: a new approach to optimization,” *Logical Methods in Computer Science*, vol. 7, no. 1, Mar. 2011. [Online]. Available: <https://doi.org/10.48550/arXiv.1012.1802>
- [2] M. Willsey, C. Nandi, Y. R. Wang, O. Flatt, Z. Tatlock, and P. Panchekha, “egg: Fast and Extensible Equality Saturation,” in *Proceedings of the ACM on Programming Languages*, vol. 5, no. POPL, art. no. 23, pp. 1-29, Jan. 2021. [Online]. Available: <https://doi.org/10.1145/3434304>
- [3] P. Panchekha, A. Sanchez-Stern, J. R. Wilcox, and Z. Tatlock, “Automatically improving accuracy for floating point expressions,” *ACM SIGPLAN Notices*, vol. 50, no. 6, pp. 1-11, Jun. 2015. [Online]. Available: <https://doi.org/10.1145/2813885.2737959>
- [4] Y. R. Wang, S. Hutchison, J. Leang, B. Howe, and D. Suciu, “SPORES: sum-product optimization via relational equality saturation for large scale linear algebra,” *Proceedings of the VLDB Endowment*, vol. 13, no. 11, pp. 1919–1932, Jul. 2020. [Online]. Available: <https://doi.org/10.14778/3407790.3407799>
- [5] J. Merckx, A. Lopoukhine, S. Coward, J. Cheng, B. De Sutter, and T. Grosser, “E-Graphs as a Persistent Compiler Abstraction,” *arXiv preprint arXiv:2602.16707*, Feb. 2026. [Online]. Available: <https://doi.org/10.48550/arXiv.2602.16707>
- [6] C. Fallin, “The acyclic e-graph: Cranelift’s mid-end optimizer,” Apr. 2026. [Online]. Available: <https://cfallin.org/blog/2026/04/09/aegraph/>